Name: Shamrudha Varshini P
Roll No: 231901048

Experiment No.: 6

Implementing Fabric Channels for Privacy

Aim

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

A Fabric Channel is a private communication mechanism that allows a subset of organizations to maintain a separate ledger and execute transactions confidentially. Each channel has its own blockchain ledger, smart contracts (chaincode), and world state database.

Organizations that are not members of a channel cannot access its ledger or transaction data. Channels enable enterprise applications to maintain confidentiality while sharing the same Hyperledger Fabric network.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

```
fabric@test-network:~$ cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

```
fabric@test-network:~$ ./network.sh up createChannel -ca
Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
Network Started Successfully
```

Step 3: Verify the Existing Channel

```
peer channel list
```

```
fabric@test-network:~$ peer channel list
Channels peers has joined:
mychannel
```

Step 4: Create a New Private Channel

```
./network.sh createChannel -c privatechannel
```

```
fabric@test-network:~$ ./network.sh createChannel -c privatechannel
Channel 'privatechannel' created successfully.
```

Step 5: Verify Available Channels

```
peer channel list
```

```
fabric@test-network:~$ peer channel list
Channels peers has joined:
mychannel
privatechannel
```

Step 6: Deploy Chaincode on the Private Channel

```
./network.sh deployCC \
-c privatechannel \
-ccn basic \
-ccl javascript \
-ccp ../asset-transfer-basic/chaincode-javascript
```

```
fabric@test-network:~$ ./network.sh deployCC \
-c privatechannel \
-ccn basic \
-ccl javascript \
-ccp ../asset-transfer-basic/chaincode-javascript
Packaging Chaincode
Installing Chaincode
Approving Chaincode
Committing Chaincode
Chaincode committed successfully
```

Step 7: Initialize the Ledger

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C privatechannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'
```

```
fabric@test-network:~$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C privatechannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'
Ledger Initialized Successfully
```

Step 8: Query Assets from the Private Channel

```
peer chaincode query \
-C privatechannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```

```
fabric@test-network:~$ peer chaincode query \
-C privatechannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
[
  {"ID":"asset1","Owner":"Tomoko","Color":"Blue"},
  {"ID":"asset2","Owner":"Brad","Color":"Red"}
]
```

Step 9: Verify Channel Isolation

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
```

```
fabric@test-network:~$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
Asset data returned only from mychannel.
Data stored in privatechannel is not accessible through mychannel.
```

Step 10: Stop the Fabric Network

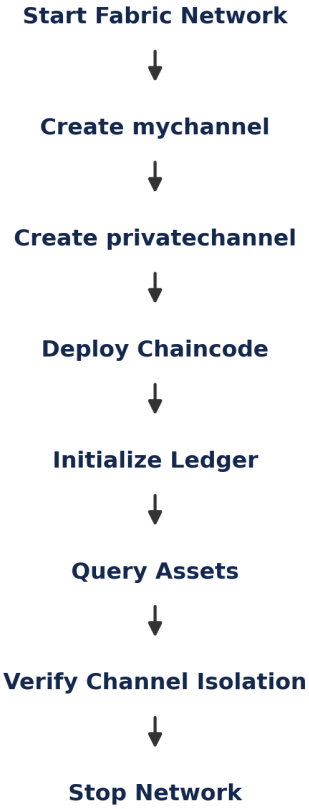
```
./network.sh down
```

```
fabric@test-network:~$ ./network.sh down
Network Stopped Successfully
```

Architecture Diagram

```
Hyperledger Fabric Network
    |
    |-- Channel 1 (mychannel) --> Peer0 Org1, Peer0 Org2 --> Ledger A
+ World State A --> Transactions A
    |-- Channel 2 (privatechannel) --> Peer0 Org1, Peer0 Org2 -->
Ledger B + World State B --> Transactions B
```

Flow Diagram



Result

The Hyperledger Fabric network was successfully configured with multiple channels. A private channel was created, chaincode was deployed, and ledger transactions were executed successfully. The experiment demonstrated that channel members can access only the data belonging to their respective channels, ensuring privacy and confidentiality in a permissioned blockchain network.